System Analysis and Design

Lec 4

How to use UML diagrams to design software system?

- Types of UML Diagrams:
  - Use Case Diagram
  - Class Diagram
  - Sequence Diagram
  - State Diagram

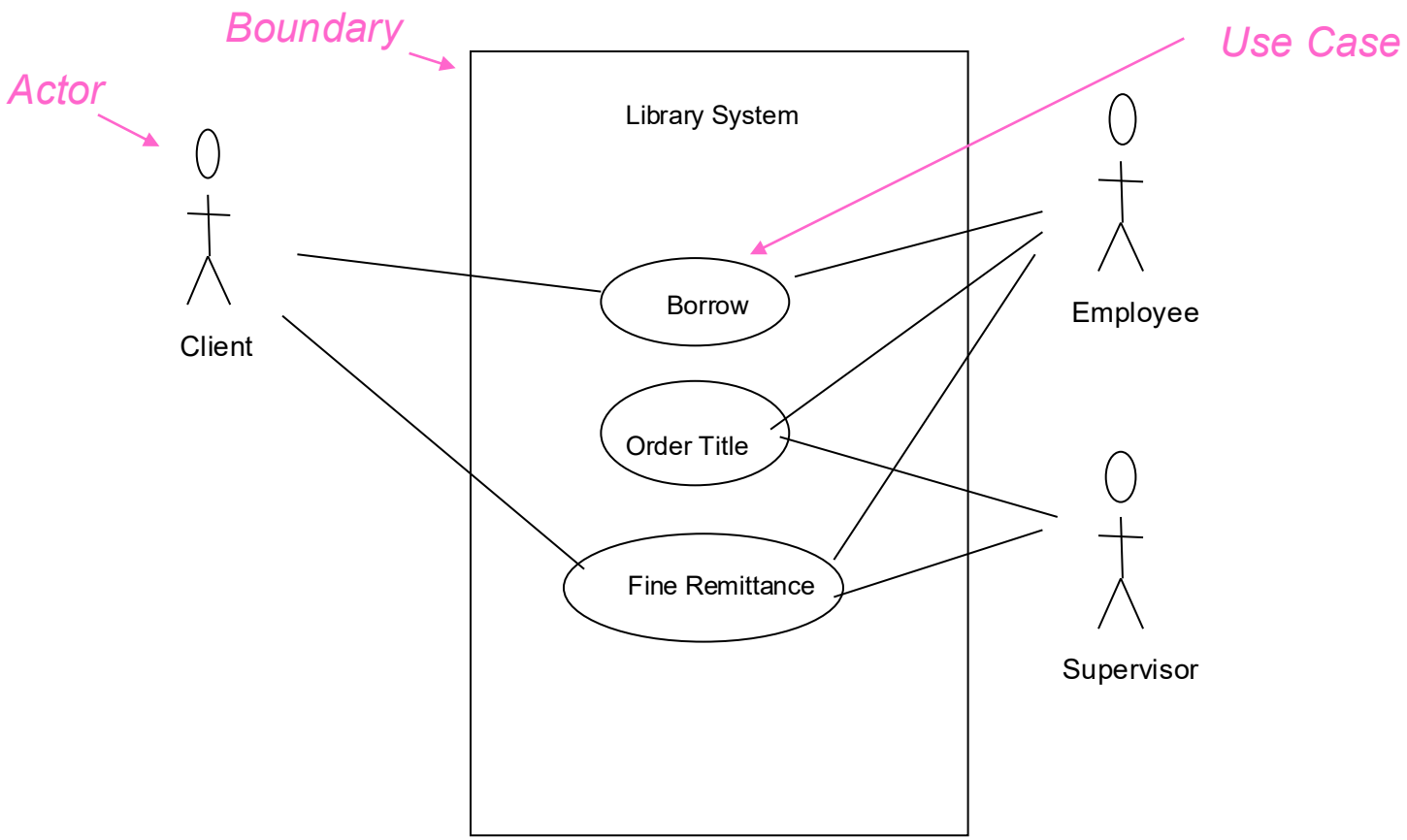
This is only a subset of diagrams … but are most widely used

Use-Case Diagrams

- A use-case diagram is a set of use cases
- A use case is a model of the interaction between
  - External users of a software product (actors) and
  - The software product itself
  - More precisely, an actor is a user playing a specific role

- describing a set of user **scenarios**
- capturing user requirements
- **contract** between end user and software developers

Use-Case Diagrams

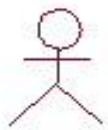


Use-Case Diagrams

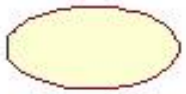
- **Actors:** A role that a user plays with respect to the system, including human users and other systems. e.g., inanimate physical objects (e.g. robot); an external system that needs some information from the current system.

- **Use case:** A set of scenarios that describing an interaction between a user and a system, including alternatives.

- **System boundary**: rectangle diagram representing the boundary between the actors and the system.



Actor



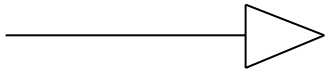
Use Case

Use-Case Diagrams

- <u>Association:</u>

communication between an actor and a use case; Represented by a solid line.

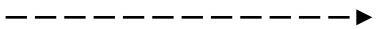
- <u>Generalization</u>: relationship between one general use case and a special use case (used for defining special alternatives) Represented by a line with a triangular arrow head toward the parent use case.



Use-Case Diagrams

<u>Include</u>: a dotted line labeled <<include>> beginning at base use case and ending with an arrows pointing to the include use case. The include relationship occurs when a chunk of behavior is similar across more than one use case. Use "include" in stead of copying the description of that behavior.

<<include>>



<u>Extend</u>: a dotted line labeled <<extend>> with an arrow toward the base case. The extending use case may add behavior to the base use case. The base class declares "extension points".

<<extend>>

Use-Case Diagrams

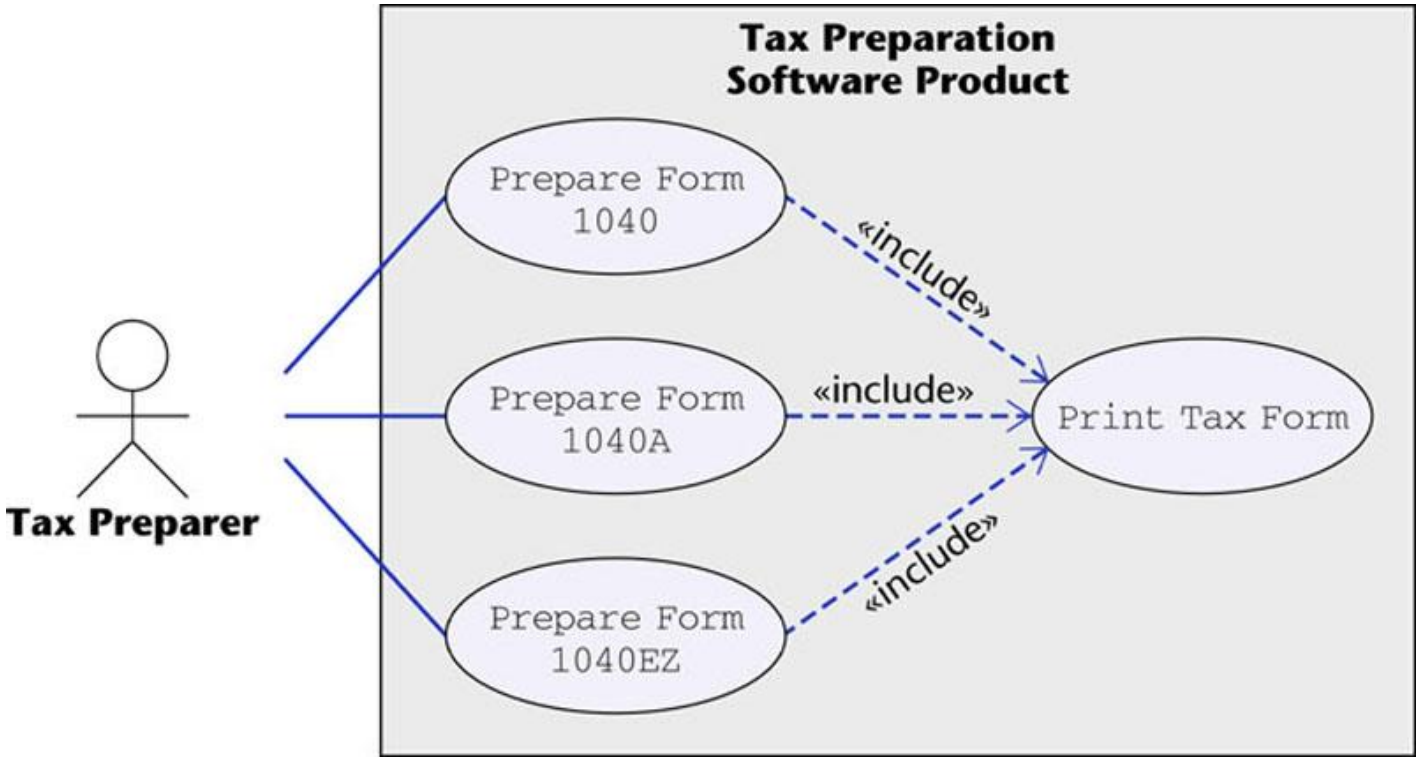
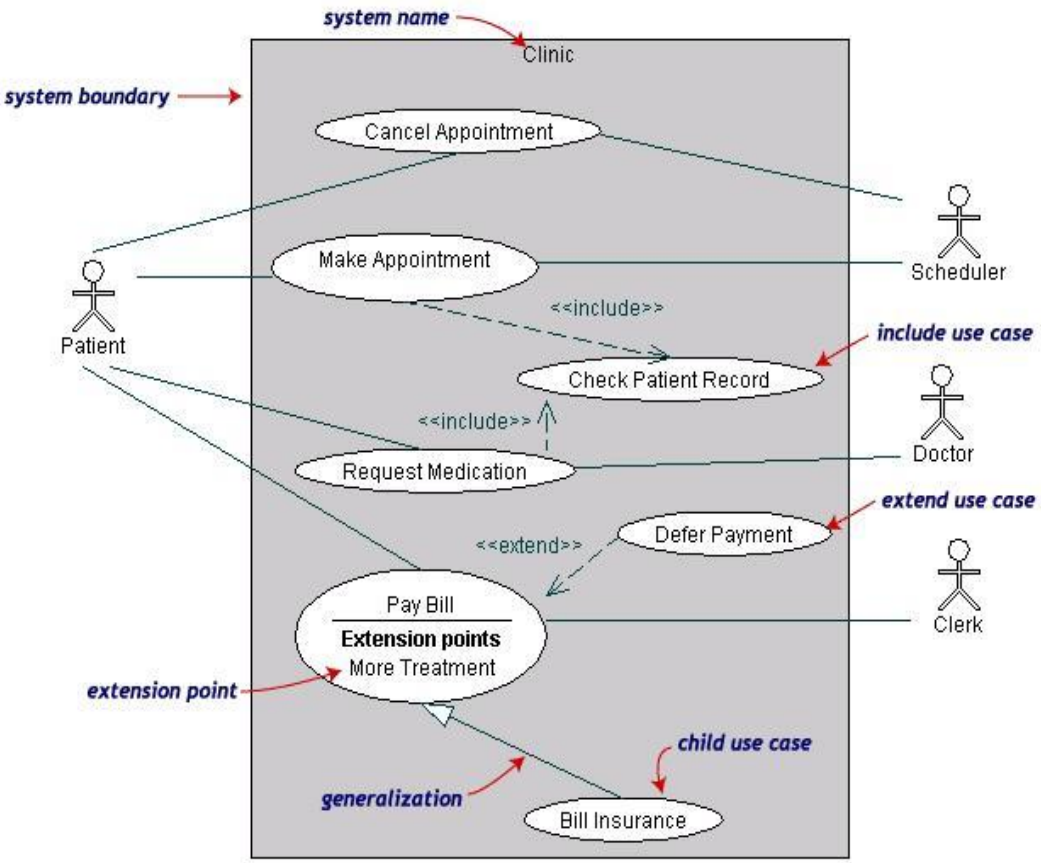


Figure 16.12

The McGraw-Hill Companies, 2005

Use-Case Diagrams

- Both **Make Appointment** and **Request Medication** include **Check Patient Record** as a subtask (include)
- The **extension point** is written inside the base case **Pay bill**; the extending class **Defer payment** adds the behavior of this extension point. (extend)
- **Pay Bill** is a parent use case and **Bill Insurance** is the child use case. (generalization)



(TogetherSoft, Inc)

Class diagram

- A class diagram depicts classes and their interrelationships
- Used for describing structure and behavior in the use cases
- Provide a conceptual model of the system in terms of entities and their relationships
- Used for requirement capture, end-user interaction
- Detailed class diagrams are used for developers

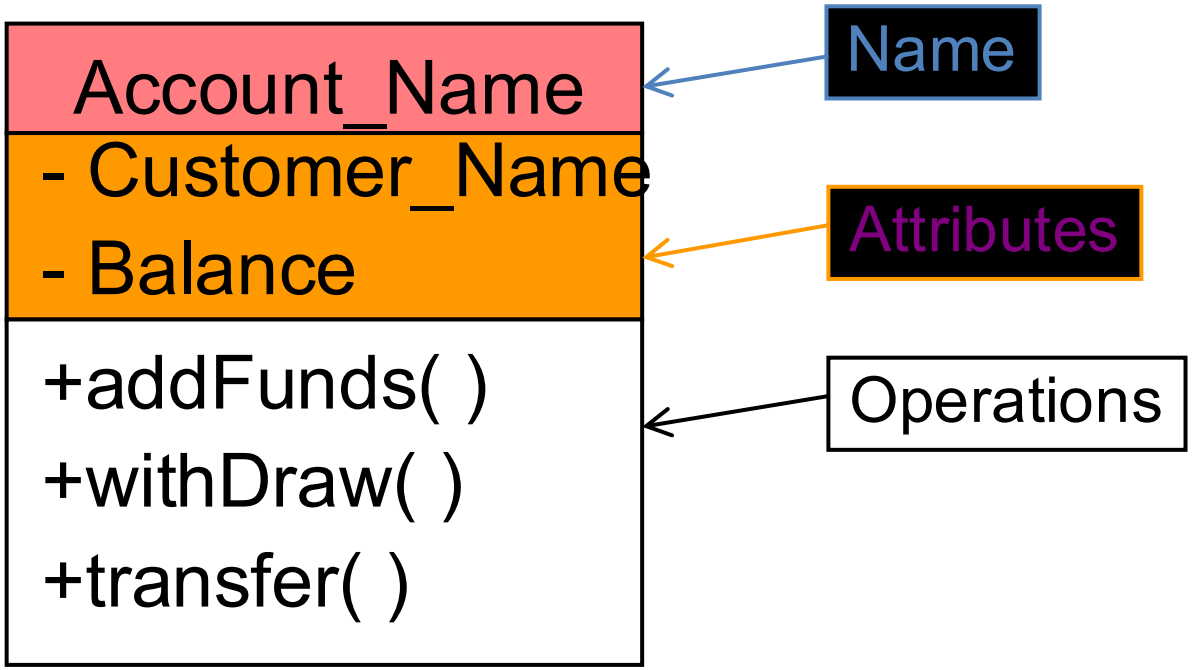
Class diagram

- Each class is represented by a rectangle subdivided into three compartments
  - Name
  - Attributes
  - Operations

- Modifiers are used to indicate visibility of attributes and operations.
  - '+' is used to denote *Public* visibility (everyone)
  - '#' is used to denote *Protected* visibility (friends and derived)
  - '-' is used to denote *Private* visibility (no one)

- By default, attributes are hidden and operations are visible.

Class diagram

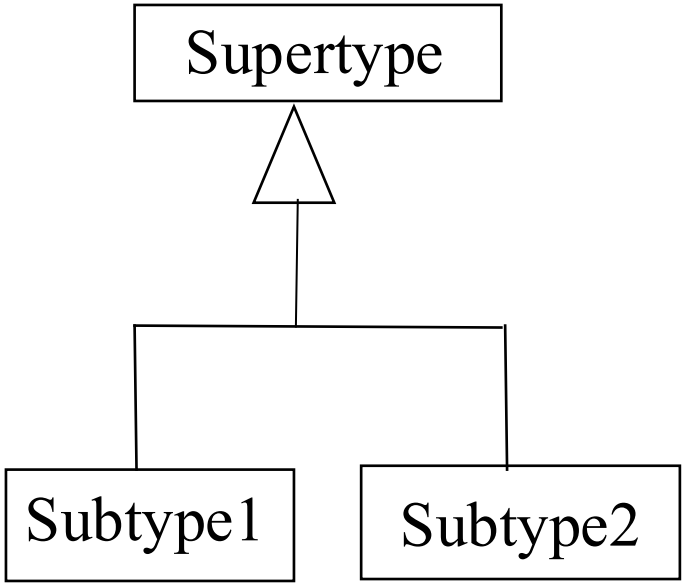


OO Relationships

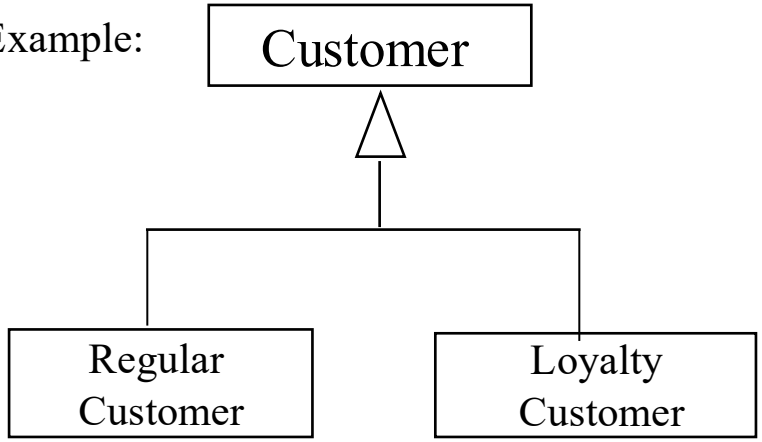
- There are two kinds of Relationships
  - Generalization (parent-child relationship)
  - Association (student enrolls in course)

- Associations can be further classified as
  - Aggregation
  - Composition

OO Relationships: **Generalization**



Example:



-Inheritance is a required feature of object orientation

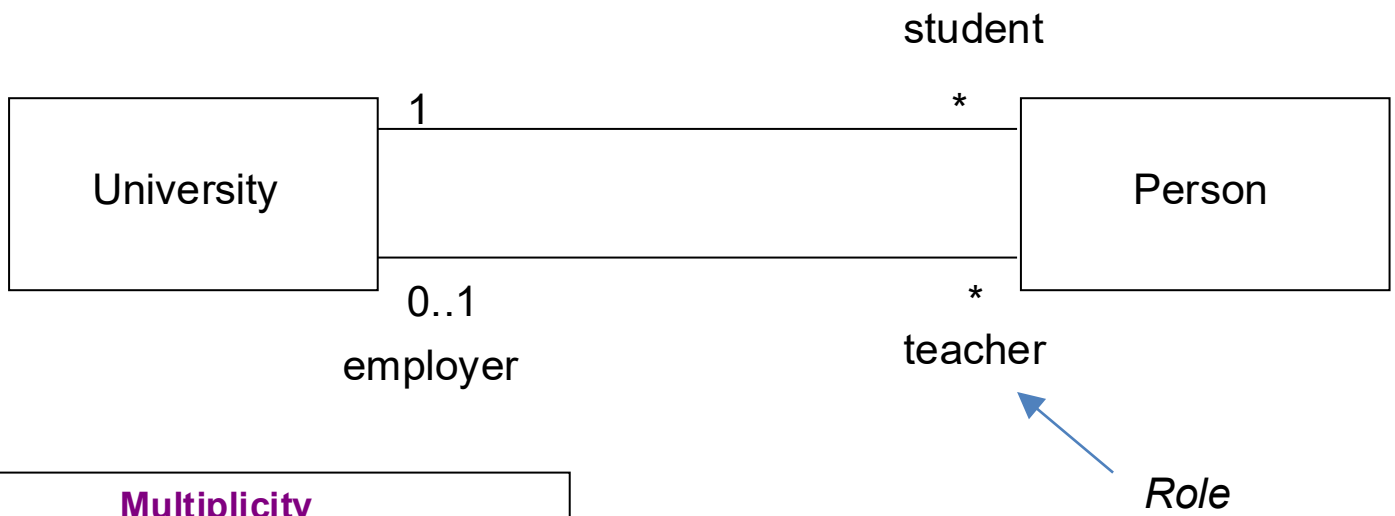
-Generalization expresses a parent/child relationship among related classes.

-Used for abstracting details in several layers

OO Relationships: **Association**

- Represent relationship between instances of classes
  - Student enrolls in a course
  - Courses have students
  - Courses have exams
  - Etc.
- Association has two ends
  - Role names (e.g. enrolls)
  - Multiplicity (e.g. One course can have many students)
  - Navigability (unidirectional, bidirectional)

Association: Multiplicity and Roles



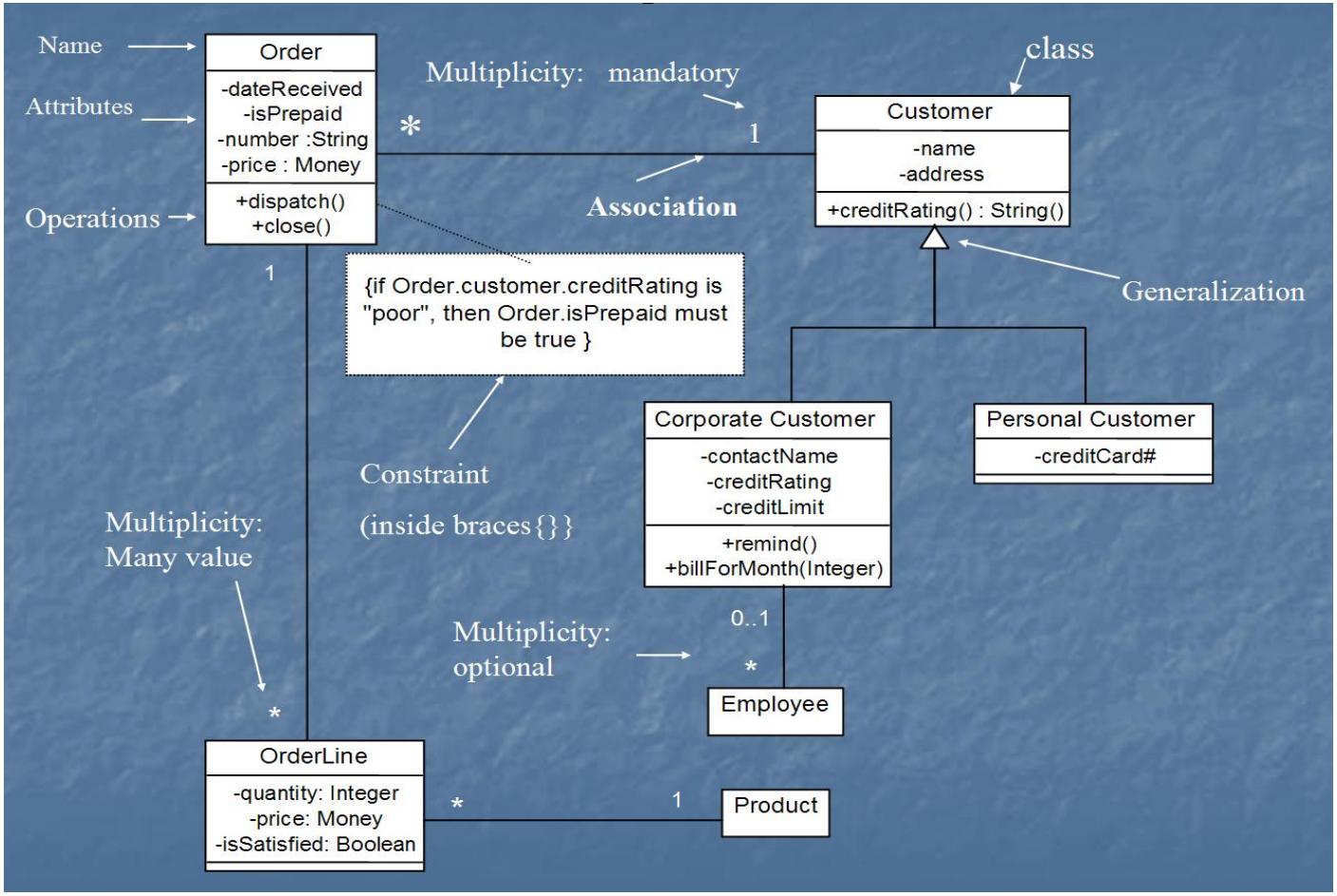
Multiplicity

Symbol	Meaning
1	One and only one
0..1	Zero or one
M..N	From M to N (natural language)
*	From zero to any positive integer
0..*	From zero to any positive integer
1..*	From one to any positive integer

Role

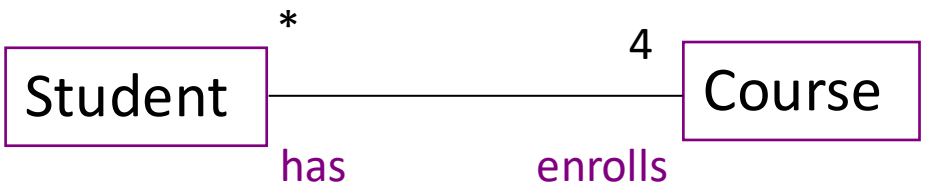
"A given university groups many people; some act as students, others as teachers. A given student belongs to a single university; a given teacher may or may not be working for the university at a particular time."

Class diagram



[from *UML Distilled Third Edition*]

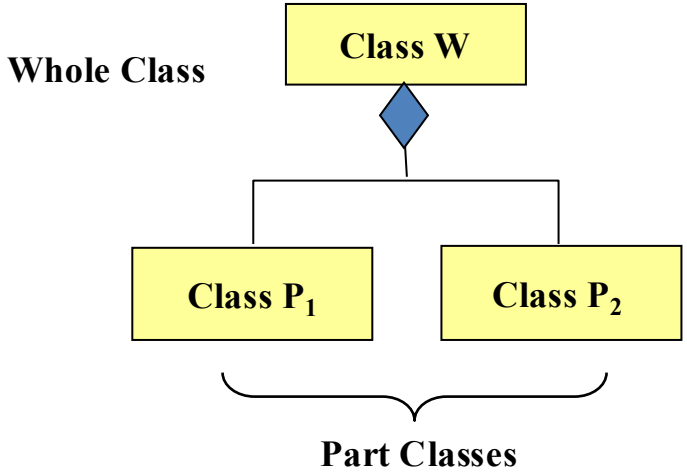
Association: Model to Implementation




```
Class Student {
   Course enrolls[4];
}

Class Course {
    Student have[];
}
```

OO Relationships: **Composition**



[From Dr.David A. Workman]

Example

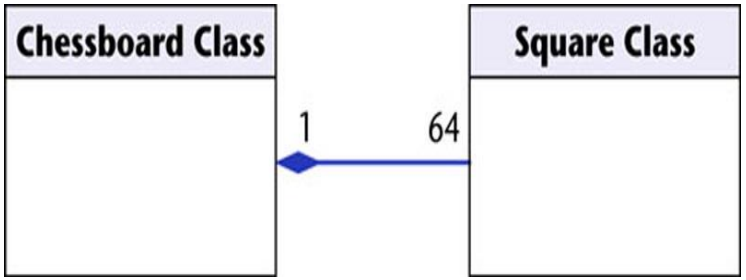


Figure 16.7

Association

Models the part–whole relationship

Composition

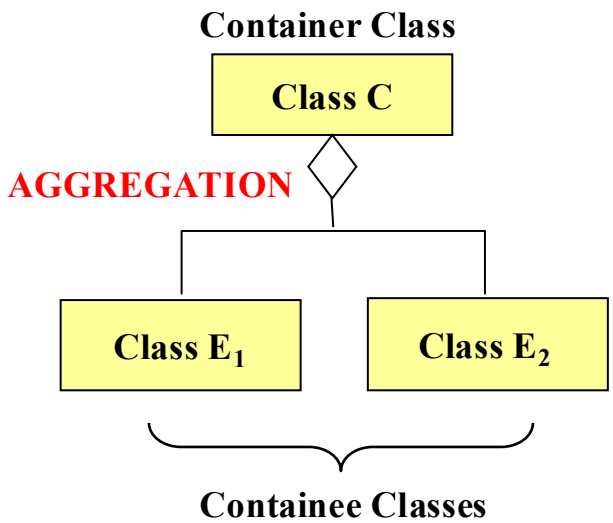
Also models the part–whole relationship but, in addition, Every part may belong to only one whole, and If the whole is deleted, so are the parts

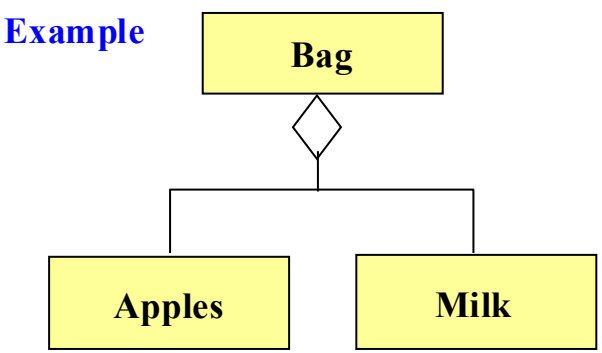
Example:

A number of different chess boards: Each square belongs to only one board. If a chess board is thrown away, all 64 squares on that board go as well.

The McGraw-Hill Companies, 2005

OO Relationships: **Aggregation**





[From Dr.David A. Workman]

Aggregation:
expresses a relationship among instances of related classes. It is a specific kind of Container-Containee relationship.

express a more informal relationship than composition expresses.

Aggregation is appropriate when Container and Containees have no special access privileges to each other.

Aggregation vs. Composition

- **Composition** is really a strong form of **association**
  - components have only one owner
  - components cannot exist independent of their owner
  - components live or die with their owner
  - e.g. Each car has an engine that can not be shared with other cars.

- **Aggregations**

may form "part of" the association, but may not be essential to it. They may also exist independent of the aggregate. e.g. Apples may exist independent of the bag.

Sequence Diagram

- A sequence diagram is a type of interaction diagram because it describes how—and in what order—a group of objects works together. These diagrams are used by software developers and business professionals to understand requirements for a new system or to document an existing process. Sequence diagrams are sometimes known as event diagrams or event scenarios.

Benefits of sequence diagrams

Sequence diagrams can be useful references for businesses and other organizations. Try drawing a sequence diagram to:

- Represent the details of a UML use case.
- Model the logic of a sophisticated procedure, function, or operation.
- See how objects and components interact with each other to complete a process.
- Plan and understand the detailed functionality of an existing or future scenario

Basic symbols and components

- To understand what a sequence diagram is, you should be familiar with its symbols and components. Sequence diagrams are made up of the following icons and elements

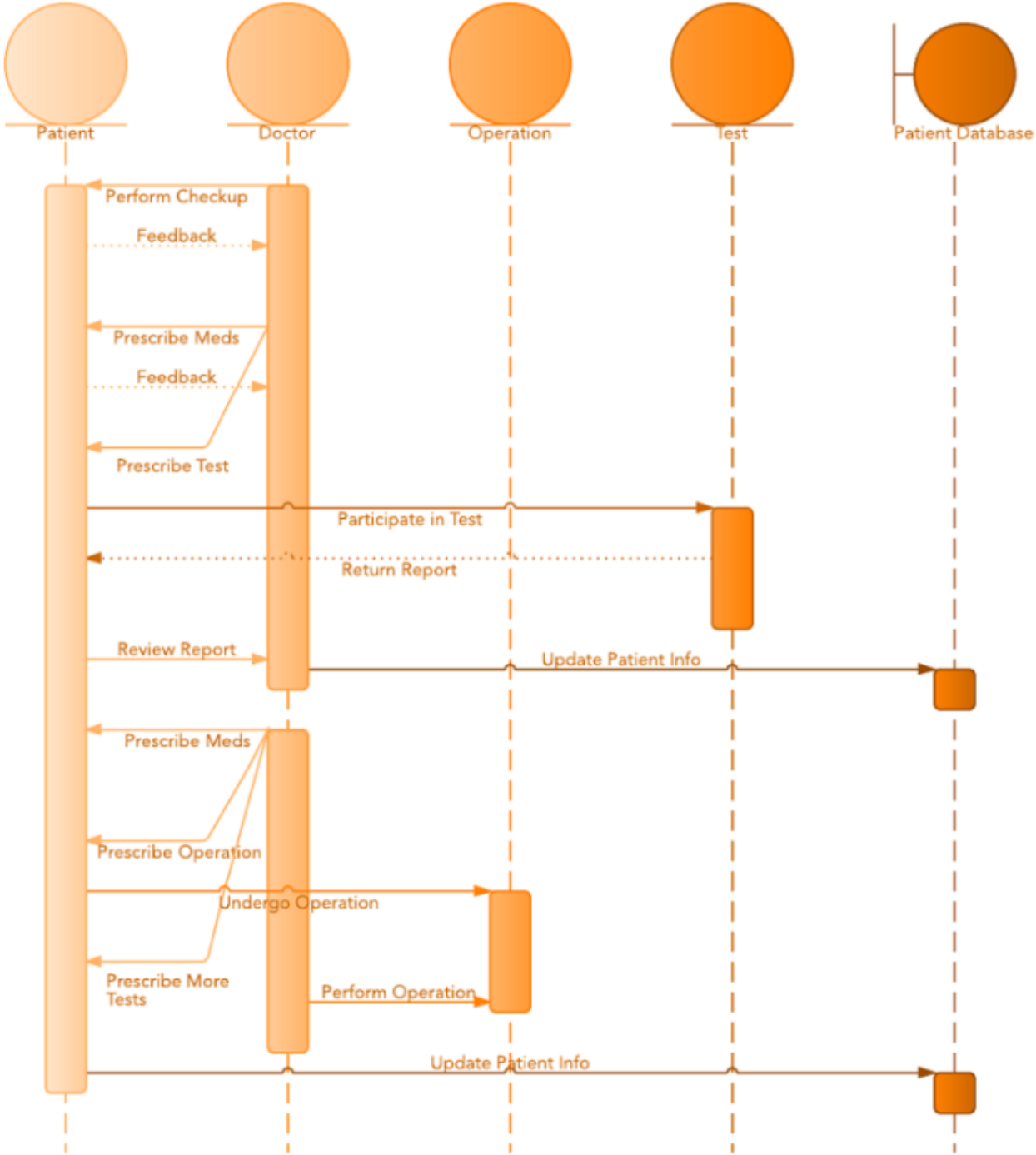
Common message symbols

- Use the following arrows and message symbols to show how information is transmitted between objects. These symbols may reflect the start and execution of an operation or the sending and reception of a signal.

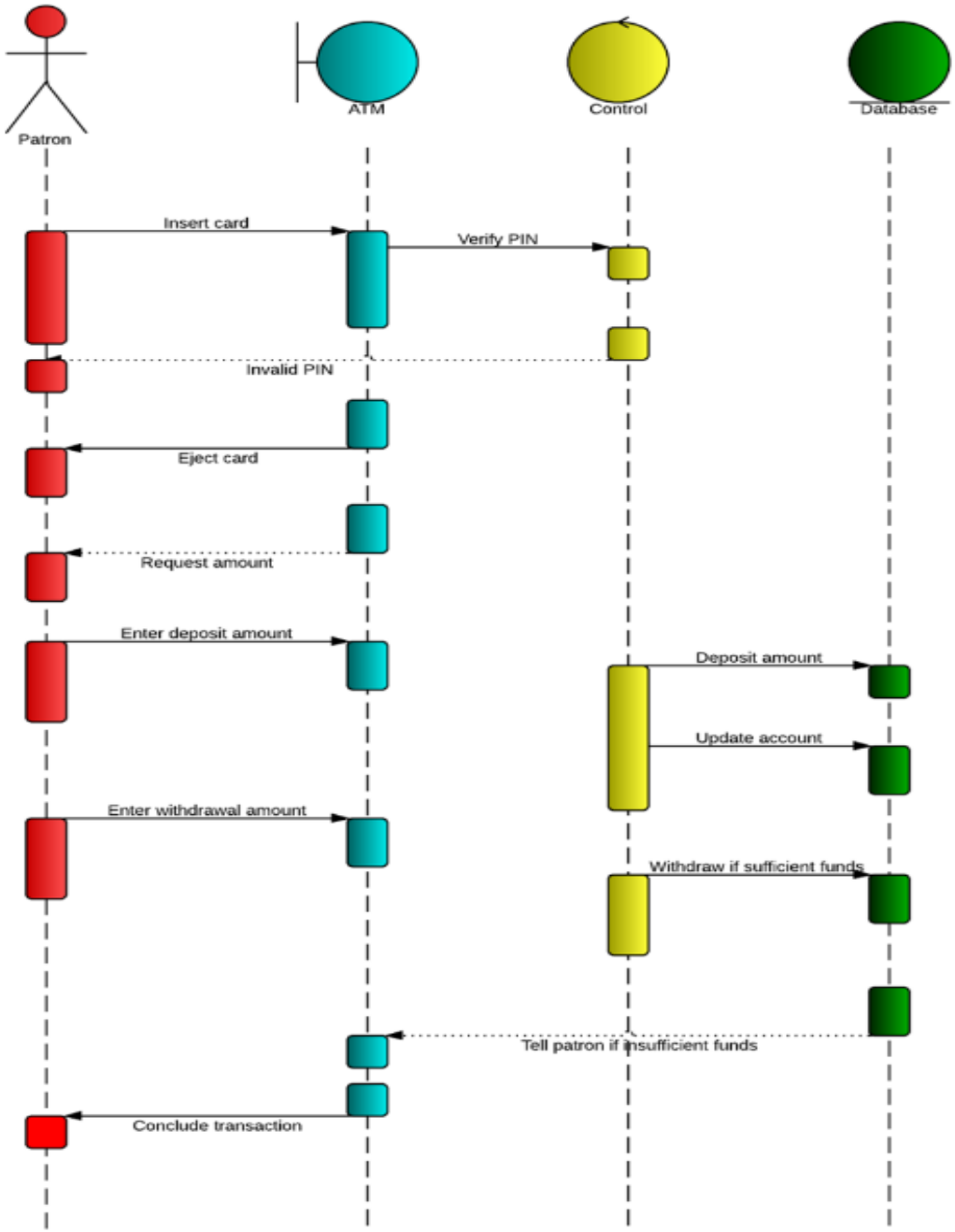
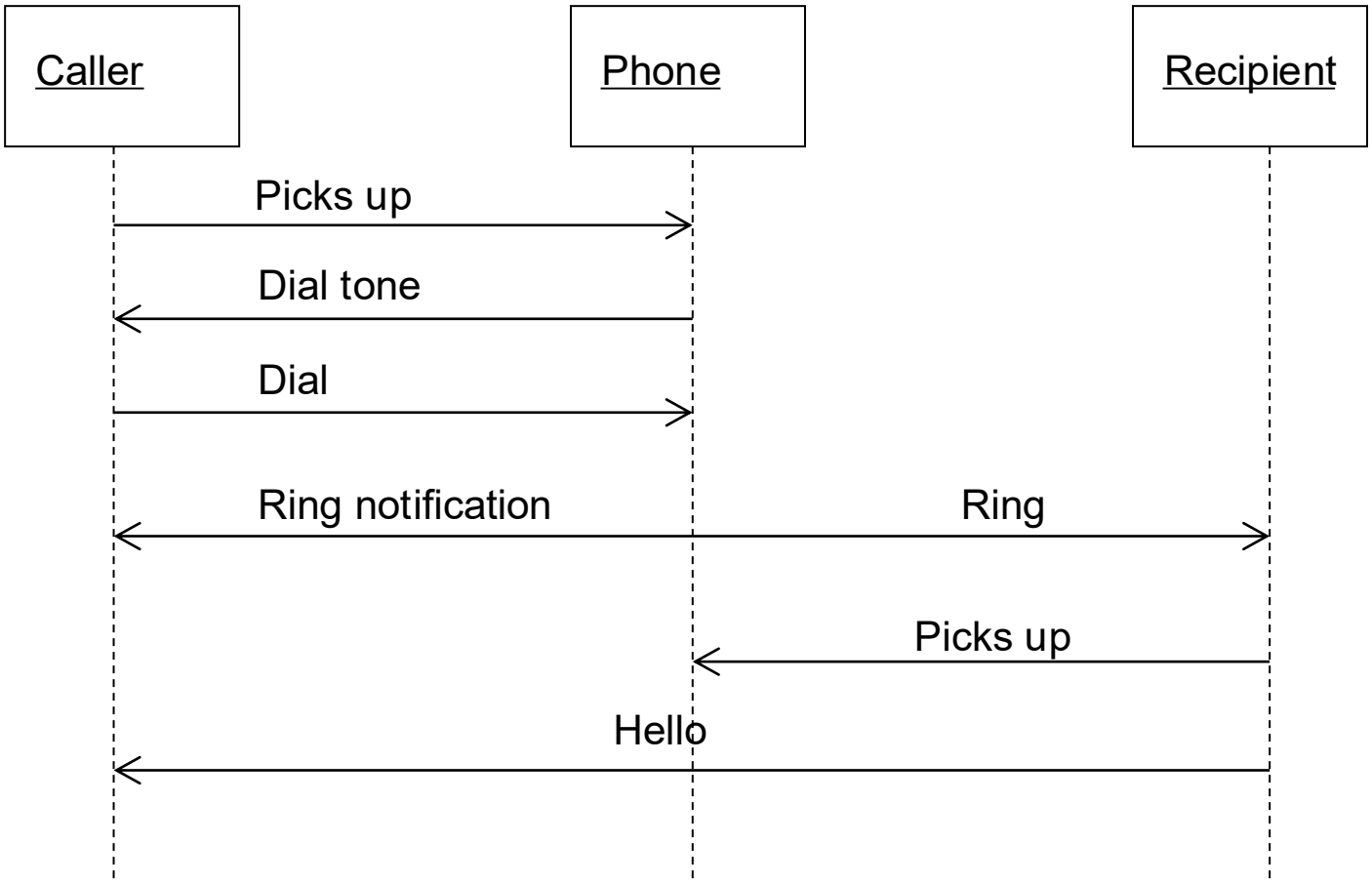
Symbol	Name	Description
	Synchronous message symbol	Represented by a solid line with a solid arrowhead. This symbol is used when a sender must wait for a response to a message before it continues. The diagram should show both the call and the reply.
	Asynchronous message symbol	Represented by a solid line with a lined arrowhead. Asynchronous messages don't require a response before the sender continues. Only the call should be included in the diagram.
	Asynchronous return message symbol	Represented by a dashed line with a lined arrowhead.
<<create>>	Asynchronous create message symbol	Represented by a dashed line with a lined arrowhead. This message creates a new object.
	Reply message symbol	Represented by a dashed line with a lined arrowhead, these messages are replies to calls.
	Delete message symbol	Represented by a solid line with a solid arrowhead, followed by an X. This message destroys an object.

Sequence diagram examples

- **Sequence diagram of a hospital management system**
- Technology has completely transformed the field of medicine, as it has with most industries. A hospital information system, also known as a hospital information system, helps doctors, administrators, and hospital staff managing all of the activities and information collected at a hospital, including checkups, prescriptions, appointments, and information on the patients and their caretakers. The diagram below provides a simple view of how the primary processes operate with each other over time.


Patient
Doctor
Operation
Test
Patient Database
Perform Checkup
Feedback
Prescribe Meds
Feedback
Prescribe Test
Participate in Test
Return Report
Review Report
Update Patient Info
Prescribe Meds
Prescribe Operation
Undergo Operation
Prescribe More Tests
Perform Operation
Update Patient Info

- **Sequence diagram for ATM systems**
- An ATM allows patrons to access their bank accounts through a completely automated process. You can examine the steps of this process in a manageable way by drawing or viewing a sequence diagram. The example below outlines the sequential order of the interactions in the ATM system.


Patron
ATM
Control
Database
Insert card
Verify PIN
Invalid PIN
Eject card
Request amount
Enter deposit amount
Deposit amount
Update account
Enter withdrawal amount
Withdraw if sufficient funds
Tell patron if insufficient funds
Conclude transaction

Sequence Diagram(make a phone call)